Evidence-aware Fit/Gap

Case Study: Fullstack Product Engineer · Submission report

Pull Request: *[paste link here once the branch is published]*
Video walkthrough (3-5 min): *[paste link here]*
Branch: feat/evidence-aware-fitgap · small commits, one per change · web: 26 Vitest tests · api: 67 RSpec examples (CI green on Ruby 3.3.2) · also run end to end on the real stack

Branch: `feat/evidence-aware-fitgap` (not published yet). Local Windows setup notes: Ruby 3.3 + DevKit, PostgreSQL, `tzinfo-data` gem, env vars ALLOWED_ORIGINS/APP_BASE_URL are required at boot. Status of each proof item is marked **DONE** or **TODO**.

Step 2. Context (what I took from the product, domain and law)

- **Users.** Assessors run and review interviews; recruiters and hiring managers read the Fit/Gap report to decide who goes to the next round. They read it between meetings, so the top of the page must answer "how much can I trust this?" before "who wins?".
- **Industry (Indonesia).** Screening volume is high and interview time is scarce; CV parsing and video-interview tooling are commoditised. The scarce thing is a *defensible, evidence-backed* reading of a skill level. Leverage sits in honesty about uncertainty, not in a prettier score.
- **Candidates (never chose this).** A wrong "gap" can end an application. UU PDP No. 27/2022 gives data subjects rights around automated processing and requires data minimisation and security. Design consequences: no AI "hire/reject" recommendation, no candidate quotes or names sent to the model for the narrative, no model error text in logs, strict tenant isolation, a visible human-override path.
- **Assumptions (no live Q&A).** Levels 1..5 are the only scale; a skill missing from the portfolio means "not covered", not "weak"; `low` AI confidence is a reason to verify, not to reject; one organisation never needs to see another's candidates.

Step 3. Findings (P0..P3)

Type: **D** = defective implementation (defined, broken). **S** = missing specification (never defined).

#	Sev	Type	Service	Finding	Impact
1	P0	D	api	`Portfolio.find(params[:id])` and `PortfolioSkill.find` have no tenant scope (Portfolio has no tenant_id). Any assessor of any org can read/regenerate/export another org's candidate fit/gap, and override another org's ratings.	Cross-tenant disclosure of candidate data (UU PDP) and tampering with another org's hiring evidence.
2	P0	D	api+web	Seam bug: API sends `expected_level`, UI reads `required_level` (and `is_override` is never sent). The "Required" column is	Assessors cannot see what the gap is measured

#	Sev	Type	Service	Finding	Impact
				blank/ `undefined` for every row and the pencil marker never shows.	against, and cannot see which ratings a human changed.
3	P1	S	api+web	"not_assessed" is excluded from the summary and nothing reports coverage. A candidate probed on 1 of 8 required skills looks the same as one probed on all 8.	Partial interviews read as complete verdicts.
4	P1	S	api+web	Low-confidence AI ratings are shown as firm "Gap".	A lightly probed skill becomes a rejection reason.
5	P1	D	api	Prompt asks the model for an "overall hiring recommendation".	Automated hiring advice about a person; weak against UU PDP expectations.
6	P1	D	api	Gemini failure is swallowed and a canned sentence is saved as a finished report forever; UI shows it under the "Culture" heading.	Silent wrong content, no retry path, mislabelled.
7	P1	D	api+web	Override destroys every report then re-enqueues; GET in the gap returns 404; UI reacts by POSTing again. Duplicate jobs; `find_or_initialize_by` then races on the unique index.	Flicker to empty, duplicate AI calls, 500s on the race.
8	P1	D	web	Polling never stops and swallows errors; non-404 failures render a blank page; export failures are silent.	Infinite spinner / empty screen.
9	P2	D	api	Duplicate vacancy skills collapse arbitrarily (`index_by`).	Lenient requirement may win over the strict one.
10	P2	D	web	Discovered skills show a raw number ("3 (confirmed)"); labels use emoji only; table does not collapse on phones.	Hard to read, poor accessibility.
11	P2	D	api	`assessor_notes` unbounded.	Oversized free text stored and exported.
14	P0	D	api	The app cannot boot in production: eager loading fails (`AudioWebsocketMiddleware` vs the real class `AudioWebSocketMiddleware`) and `config.active_job` is called while Active Job is not loaded. Found only when CI eager-loaded the app and when booting with RAILS_ENV=production.	Nothing can be released; dev mode hides it.
12	P3	S	both	No tests, no CI, no test runner on web.	Nothing protects any of the above.
13	P1	D	api	PDF export raises 500 on any character outside Windows-1252: it prints "→" itself for every overridden skill, and candidate quotes, role titles or AI text with emoji or non-Latin characters do the same. Found by running the real stack, not by reading code.	Export fails exactly for the candidates a human reviewed.

Constraint signal (escalate to Tech Lead).

1. Tenancy is by convention (`TenantScoped` on some models, none on Portfolio/FitGapReport); finding 1 is a class of bug, not one bug. Needs a decision: tenant_id on every table vs. always going through Session.
2. All AI levels and narratives are unvalidated against ground truth; before this informs hiring we need an agreed review/consent flow for candidates (UU PDP notice, access, objection).
3. Local dev on Windows can need extra setup (tzinfo-data gem; Smart App Control blocked native Ruby extensions until the user reinstalled Ruby).

4. `db/schema.rb` and enum types live in a schema (`ai_interview`) shared with another service (rakamin-api); migrations here must stay additive.

Step 4. Strategy

Chosen slice: Fit/Gap report, end to end (data model, comparison service, API, UI, tests, CI).

Options

	A. Patch keys only	B. Deterministic comparator + split narrative (chosen)	C. Compute everything on read, cache only narrative
Impact	Fixes finding 2 only	Fixes 1-8, table never blocked by AI	Same as B; no stored comparison to go stale
Cost	Hours	~1 migration, 1 service, 1 page	Larger rewrite of export/PDF paths
Maintainability	Leaves the architecture debt	Pure class, easy to test and reason about	Fewer moving parts long term
Failure modes	Model failure still hides facts	Stored comparison can be stale; mitigated by recompute-on-override	Slower reads; AI cache invalidation still needed
Reversible	Yes	Yes (additive migration, legacy shape normalised on read)	Harder to walk back
Fit	Too small to matter	Fits the deadline and the existing storage	Best end state, too big for this deadline

I chose **B**; C is the follow-up once consumers of the stored JSON are known.

Acceptance criteria (written before code)

1. Skill with no matching portfolio entry -> `not_assessed` , never `gap` .
2. Match by `skill_id` first, then normalised label (case and whitespace insensitive); similar-but-different labels do not match.
3. Assessor override replaces the AI level everywhere (table, summary, narrative prompt, PDF) and is flagged.
4. Low confidence and not overridden -> `low_evidence` ; UI says "Gap · verify".
5. Summary always reports `assessed / total` and coverage %; empty vacancy -> `coverage_pct: null` , friendly empty state.
6. The table is returned by the first request. Only the narrative is asynchronous.
7. Model timeout/error/empty/non-JSON/wrong type -> narrative `failed` , factual fallback text, table intact, retry possible, no exception to Sidekiq, no model error text in logs.
8. Duplicate jobs for unchanged inputs are no-ops; a result computed from stale inputs is discarded; two creators of the same row do not raise.
9. An override never causes a missing report (no destroy/404 window).
10. Another tenant gets 404 on every fit/gap, export and override endpoint; cross-tenant vacancy/portfolio pairing is refused.
11. Inputs: override level must be an integer 1..5; notes <= 2000 chars; `vacancy_id` required.
12. UI states: loading, portfolio not ready, error with retry, empty vacancy, nothing assessed, partial (narrative updating / failed / timed out), long labels, phone width, export error.
13. Legacy stored reports (`expected_level`) still render.

14. Migration reversible and safe on existing rows.

Step 5. Execution proof

Item	Status
Web: typecheck, 26 Vitest tests, production build	**DONE** (run locally, all green)
API: RSpec (comparator, engine, request, export, worker, migration specs)	**DONE** locally: 67 examples, 0 failures (Ruby 3.3.12, Postgres 18, migrations from scratch). Running them found a real bug (exclusion validator rejected an empty array), fixed in 38f9630. The PDF crash spec was watched failing (3 failures) without the fix and passing with it.
Full stack run per the READMEs	**DONE** on Windows: Redis (portable community build, because the Memurai MSI is blocked by policy), Sidekiq, Rails on 3001, web on 5173 with .env, db:migrate + db:seed + demo data. Verified with curl and a browser: POST fitgap returns the table immediately (202), Sidekiq runs the job, with a fake Gemini key the narrative becomes failed with a factual fallback, the log contains only the error class, override updates the table instantly with no 404 window, another tenant gets 404 on read and export, bad input gets 404/422, no token gets 403, PDF/JSON export return 200. 0 browser console errors; portfolio -> choose vacancy -> Fit/Gap flow works.
Migration applies from scratch and is reversible	Applied from scratch **DONE**; rollback + re-apply covered by migration spec (**DONE**) and a CI step; schema.rb matched the generated one.
Seeded fault test	**DONE**: branch scratch/seeded-faults, 4 faults each committed, shown failing, then reverted (see assessment/seeded-fault-log.txt): tenant scope dropped (request spec fails), unassessed->gap (6 comparator specs fail), stale-input check removed (engine spec fails), UI wrong key (3 Vitest fail). After each revert the suite is green again.
Screenshots (states, mobile, before/after, real stack)	**DONE**: assessment/screenshots. Edge states (updating, failed, nothing assessed, empty, error, loading) use stubbed API responses; real-0x-*.png come from the real stack.
3-5 min video	**TODO (author to record)**
Pull Request link	**TODO**: branch is intentionally unpublished until the author approves.

AI verification moment (real, from this work)

I generated the new `FitGapReportPage.tsx` in the same shell command as other files. The command failed to parse, so the file was never written, yet my next steps (typecheck, build) still looked healthy because the *old* page compiled. The new page tests (all 8 failing, DOM showed the old `max-w-2xl` layout) exposed it; I rewrote the file and the tests went green. Lesson applied: tests assert the new behaviour, not just compilation, and I stopped trusting "no error output" as proof of a write.

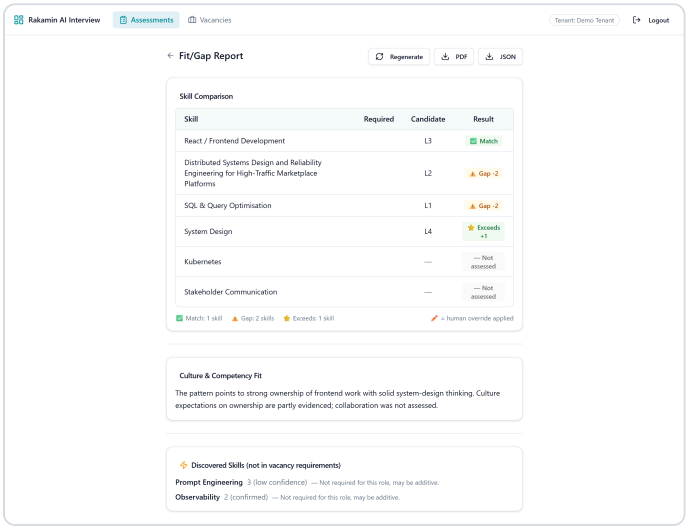
A second risk I corrected in review: my first controller draft looked up portfolios with `Portfolio.joins(:session)`, which does not reliably apply the tenant default scope of `Session` inside the join; I replaced it with `Portfolio.where(session_id: Session.select(:id))`, which does, and covered it with the cross-tenant request specs (these still need a CI run to confirm).

Known limits (honest)

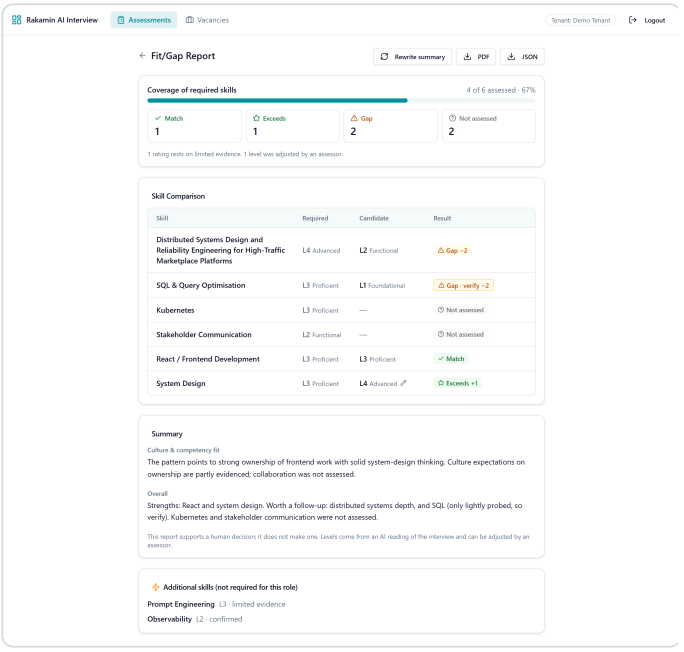
- CI itself has not run yet (needs a push); local runs use Ruby 3.3.12 instead of the pinned 3.3.2 via an untracked local Gemfile copy.
- Gemini was never called for real (no key): the "ready" narrative path is covered by specs with a fake client, the failure path was also exercised end to end.
- Windows run notes: Puma needs WEB_CONCURRENCY=0 (worker mode is unsupported), the tzinfo-data gem is needed, ALLOWED_ORIGINS and APP_BASE_URL are required at boot, api/README says ai-interview-web but the folder is web, and web/README mixes ports 3000 and 3001 (3001 is correct).
- `portfolio_skills.skill_id` / `vacancy_skills.skill_id` are free strings; matching by label is a heuristic.
- No candidate-facing notice or objection flow yet (recommended next step for UU PDP).
- Export JSON now includes `summary` and `narrative_status`; external consumers of the old shape keep `expected_level`.

Screenshots: before vs after

Same data, same vacancy. **Before** is the current `main`; **after** is this branch.

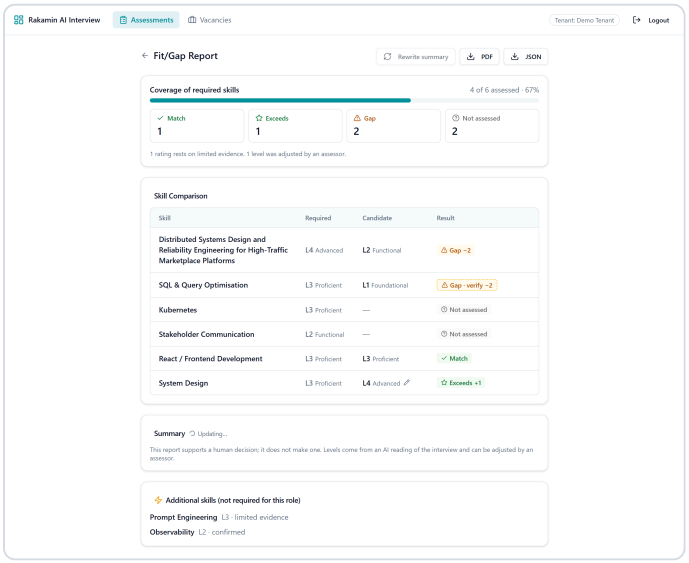


Before: Required column empty (API/UI key mismatch), no coverage, no override marker, discovered skills as raw numbers.

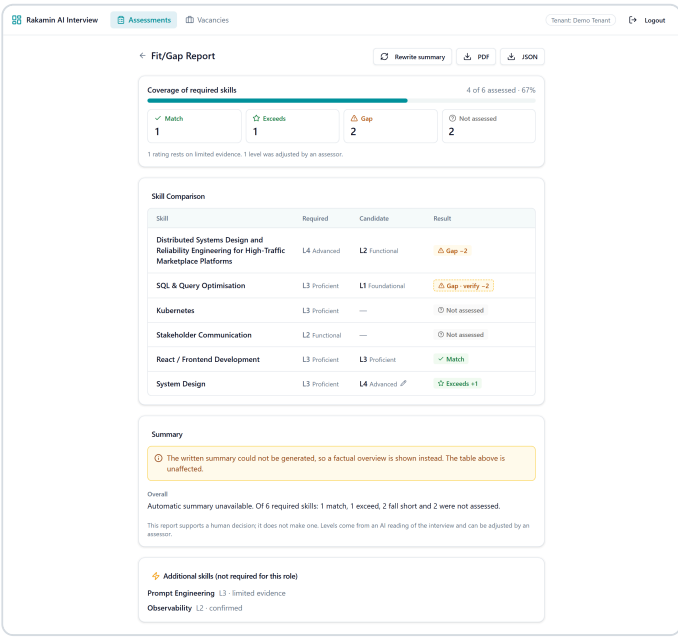


After: coverage first, required levels, gaps sorted first, low-evidence gap marked verify, override marker, separate narratives.

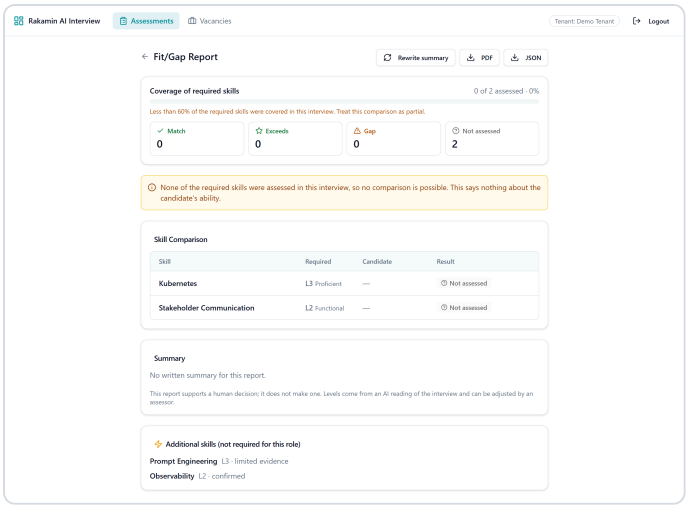
All interaction states



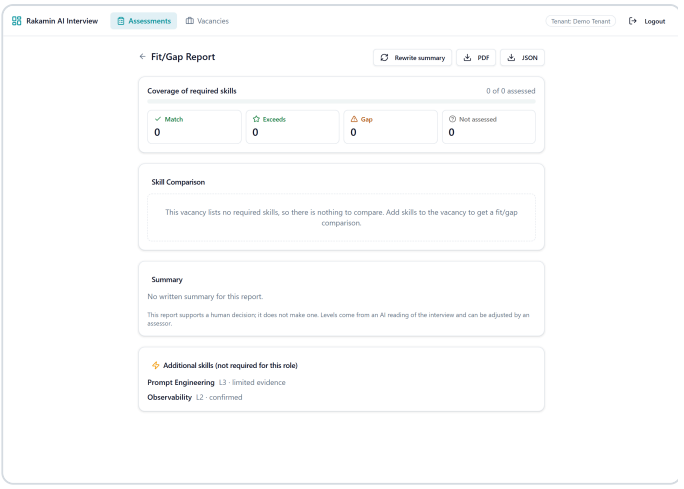
Narrative updating: table already usable



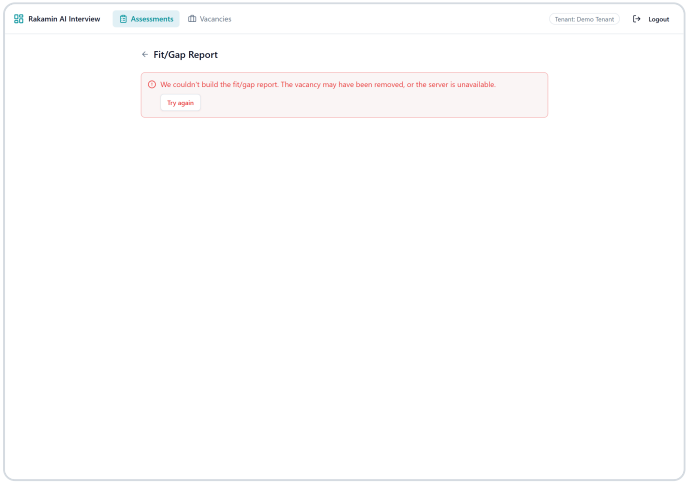
AI failed: factual fallback, table intact



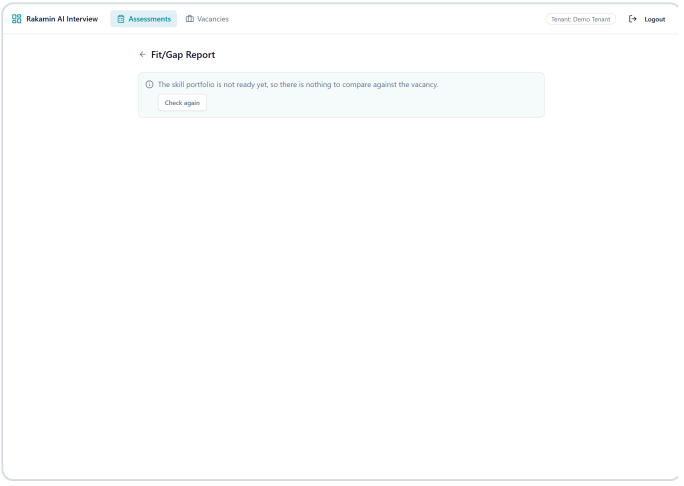
Nothing assessed: says nothing about the candidate



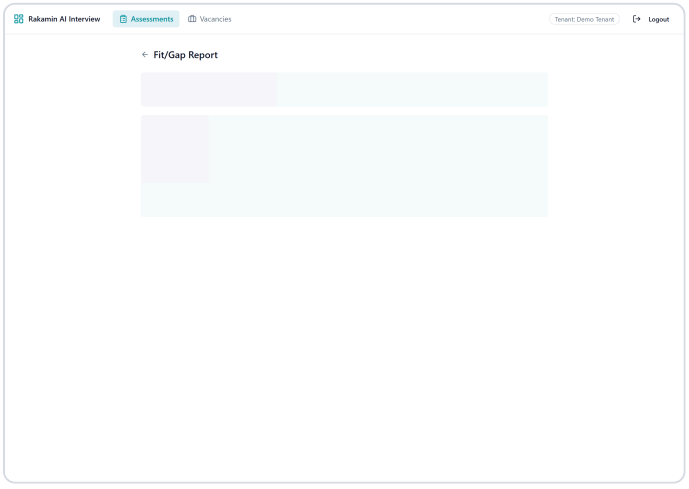
Vacancy with no required skills



Server error with retry



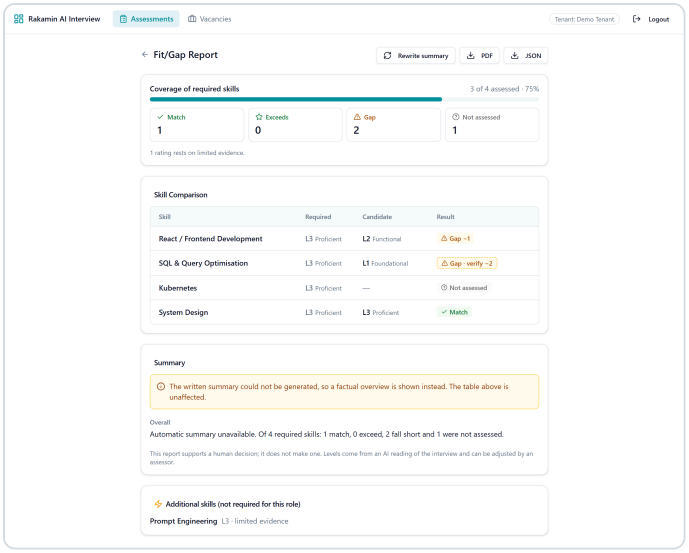
Portfolio still generating



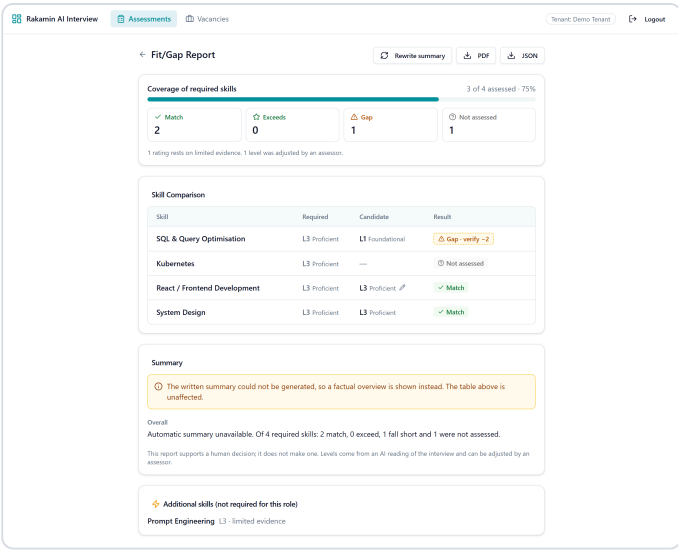
Loading skeleton

Real stack (Rails + Sidekiq + Redis + Postgres + Vite, per the READMEs)

Gemini was given a fake key, so the narrative takes the failure path. Left: first load. Right: after an assessor override of React from L2 to L3: the table updated instantly, no 404 window, summary and counts recomputed.

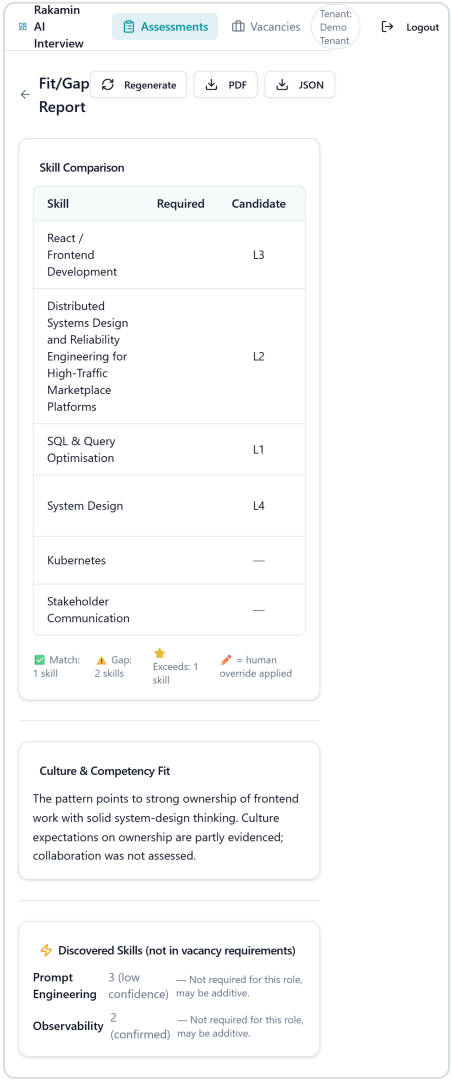


Real: first load

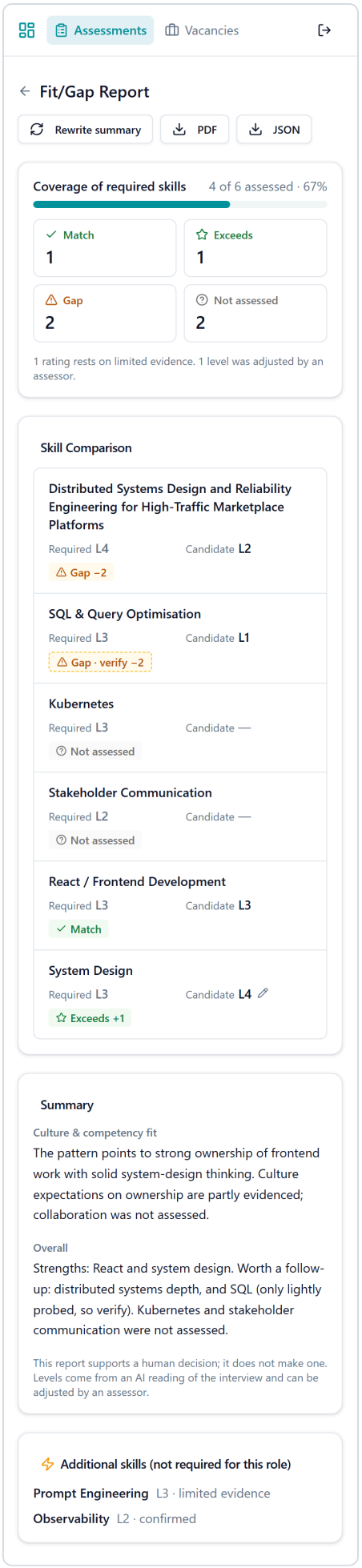


Real: after override (pencil marker, 2 matches, 1 gap)

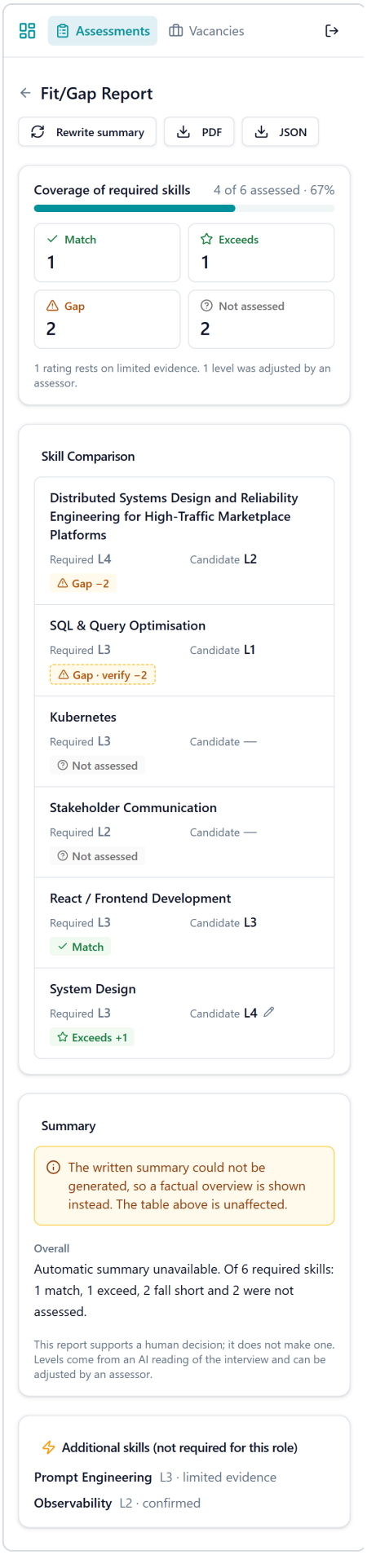
Responsive (390px wide)



Before: page overflows the viewport



After: rows stack, no horizontal scroll



After: failed narrative on phone

Appendix: seeded fault test log (raw)

```
=== FAULT: drop tenant scope in find_portfolio
 1 file changed, 1 insertion(+), 1 deletion(-)
17 examples, 1 failure
rspec ./spec/requests/fit_gap_spec.rb:78 # Fit/gap API tenant isolation (UU PDP: another organisation must never see a candidate) hides the portfolio from another tenant on every fit/gap endpoint
--- after revert:
17 examples, 0 failures
=== FAULT: unassessed skill reported as gap
 1 file changed, 1 insertion(+), 1 deletion(-)
19 examples, 6 failures
rspec ./spec/services/fit_gap/comparator_spec.rb:43 # FitGap::Comparator unassessed skills is not_assessed when the portfolio is empty
rspec ./spec/services/fit_gap/comparator_spec.rb:47 # FitGap::Comparator unassessed skills is not_assessed when the matching skill has no level
rspec ./spec/services/fit_gap/comparator_spec.rb:38 # FitGap::Comparator unassessed skills is not_assessed, never a gap, when the candidate has no matching skill
rspec ./spec/services/fit_gap/comparator_spec.rb:125 # FitGap::Comparator summary reports 0% coverage when nothing was assessed
rspec ./spec/services/fit_gap/comparator_spec.rb:114 # FitGap::Comparator summary counts every outcome and computes coverage
rspec ./spec/services/fit_gap/comparator_spec.rb:63 # FitGap::Comparator matching does not match different skills with similar text
--- after revert:
19 examples, 0 failures
=== FAULT: write narrative even when inputs changed (stale discard removed)
 1 file changed, 1 insertion(+), 1 deletion(-)
17 examples, 1 failure
rspec ./spec/services/fit_gap/engine_spec.rb:165 # FitGap::Engine#generate_narrative! discards a result computed from stale inputs when an override lands mid-call
--- after revert:
17 examples, 0 failures
=== FAULT: UI reads wrong key again (seam regression)
 1 file changed, 1 insertion(+), 1 deletion(-)
⎯⎯⎯⎯⎯⎯ Failed Tests 3 ⎯⎯⎯⎯⎯⎯
 FAIL  src/lib/fitgap.test.ts > normalizeComparison (API/UI seam) > reads legacy expected_level when required_level is missing
 FAIL  src/lib/fitgap.test.ts > normalizeComparison (API/UI seam) > prefers required_level
 FAIL  src/components/fitgap/ComparisonTable.test.tsx > ComparisonTable > shows the required level (regression: the column read a key the API never sent)
      Tests  3 failed | 23 passed (26)
--- after revert:
      Tests  26 passed (26)
2be2ffd Revert "seed fault: UI reads wrong key again (seam regression) (must be caught by the tests)"
60162ed seed fault: UI reads wrong key again (seam regression) (must be caught by the tests)
d845a34 Revert "seed fault: write narrative even when inputs changed (stale discard removed) (must be caught by the tests)"
129cf15 seed fault: write narrative even when inputs changed (stale discard removed) (must be caught by the tests)
f8f812a Revert "seed fault: unassessed skill reported as gap (must be caught by the tests)"
f1fb44b seed fault: unassessed skill reported as gap (must be caught by the tests)
66ae6d5 Revert "seed fault: drop tenant scope in find_portfolio (must be caught by the tests)"
fbfd601 seed fault: drop tenant scope in find_portfolio (must be caught by the tests)
3c1773a ci: provide ALLOWED_ORIGINS and APP_BASE_URL the app requires at boot
```